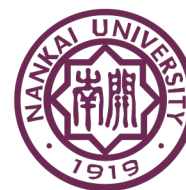


第5讲 OpenMP共享内存编程



南开大学软件学院

NANKAI UNIVERSITY
COLLEGE OF SOFTWARE



南开大学

Nankai University

■ 第五章 OpenMP共享内存编程

- 5.1 预备知识
- 5.2 梯形积分法
- 5.3 变量的作用域
- 5.4 归约子句
- 5.5 parallel for
- 5.6 更多OpenMP的循环：排序
- 5.7 循环调度
- 5.8 生产者和消费者问题

目录

1. OpenMP并行模型
2. 并行循环
3. 数据依赖
4. 循环调度
5. 局部性
6. 任务并行

目录

1. OpenMP并行模型
2. 并行循环
3. 数据依赖
- 4. 循环调度**
5. 局部性
6. 任务并行

4.1 编程模型：循环调度

■ `schedule`子句确定如何在线程间划分循环

- `static([chunk])`静态划分

- 分配给每个线程 `[chunk]`步迭代，所有线程都分配完后继续循环分配，直至所有迭代步分配完毕
- 默认`[chunk]`为`ceil(#iterations/#threads)`

- `dynmaic([chunk])`动态划分

- 分给每个线程`[chunk]`步迭代，一个线程完成任务后再为其分配`[chunk]`步迭代
- 逻辑上形成一个任务池，包含所有迭代步
- 默认`[chunk]`为1

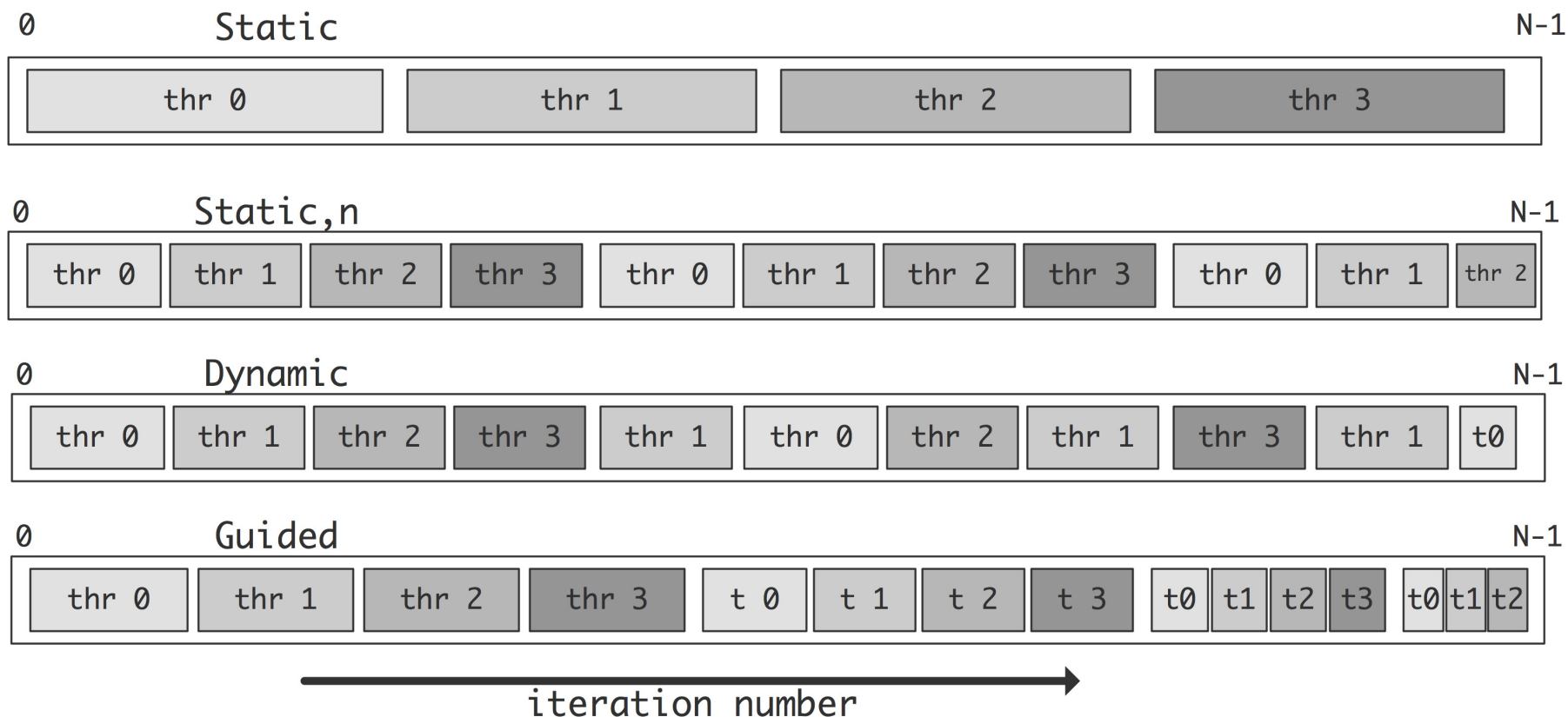
- `guided([chunk])`动态划分，但划分过程中`[chunk]`指数减小

- 类似于DYNAMIC调度，但分块开始大，随着迭代分块越来越少，循环区间的划分是基于类似下列公式完成的（不同的编译系统可能不同）：

$$S_k = \left\lceil \frac{R_k}{2N} \right\rceil$$

其中N是线程个数， S_k 表示第k块的大小， R_k 是剩余未被调度的循环迭代次数

4.1 编程模型：循环调度



<https://theartofhpc.com/pcse/omp-loop.html>

4.2 循环调度：调度字句

■ 默认调度

```
sum = 0.0;  
# pragma omp parallel for num_threads(thread_count) reduction(+:sum)  
    for (i = 0; i <= n; i++)  
        sum += f(i);
```

■ 等价于

```
sum = 0.0;  
# pragma omp parallel for num_threads(thread_count) reduction(+:sum) \  
schedule(static, n/thread_count)  
    for (i = 0; i <= n; i++)  
        sum += f(i);
```

4.3 更多循环调度属性

- **RUNTIME**: 调度决策推迟到运行时由环境变量指定
需提前设定环境变量**OMP_SCHEDULE**
- **AUTO**: 委托编译器和/或运行时系统做出调度决策
- **NO WAIT/nowait** : 线程在并行循环结束时不进行同步
- **ORDERED** : 循环步必须串行执行
- **COLLAPSE** : 指出嵌套循环如何收缩为一个大的循环并根据子句进行划分（收缩顺序依据原串行执行顺序）

4.4 OpenMP环境变量

■ OMP_NUM_THREADS

- 设置执行期间使用的线程数
- 若允许动态调整线程数，则此环境变量值为最大线程数
- 例如

```
setenv OMP_NUM_THREADS 16 [csh, tcsh]
```

```
export OMP_NUM_THREADS=16 [sh, ksh, bash]
```

■ OMP_SCHEDULE

- 应用于调度类型为RUNTIME的do/for和parallel do/for指示
- 为所有这些循环设置调度类型和块大小
- 例如

```
setenv OMP_SCHEDULE GUIDED,4 [csh, tcsh]
```

```
export OMP_SCHEDULE=GUIDED,4 [sh, ksh, bash]
```

4.5 调度例：多个不均衡数组排序

```
for (int i = 0; i < ARR_NUM; i++)  
    stable_sort(arr[i].begin(), arr[i].end());
```

串行：1969ms

```
#pragma omp parallel for num_threads(THREAD_NUM)  
for (int i = 0; i < ARR_NUM; i++)  
    stable_sort(arr[i].begin(), arr[i].end());
```

静态块划分：618ms

```
#pragma omp parallel for num_threads(THREAD_NUM) schedule(dynamic, 50)  
for (int i = 0; i < ARR_NUM; i++)  
    stable_sort(arr[i].begin(), arr[i].end());
```

动态划分：516ms

4.6 调度决策对性能的影响

■ 负载均衡

- 每步迭代工作量一致？
- 处理器计算速度一致？

■ 调度开销

- 静态决策代价很低，因为不需要运行时协调
- 动态决策：依赖于决策方法的复杂性和频率

■ 数据局部性

- 小chunk下，cache line内的局部性特别要考虑
- 也影响同一个处理器上的数据重用

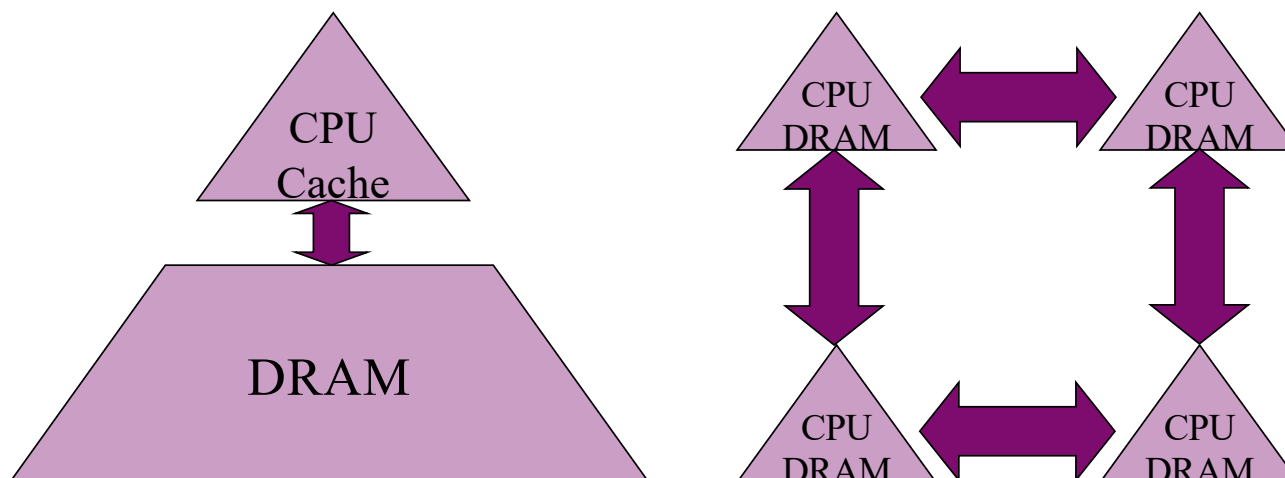
目录

1. OpenMP并行模型
2. 并行循环
3. 数据依赖
4. 循环调度
- 5. 局部性**
6. 任务并行

5.1 编程模型：循环调度

■ 算法代价分为两部分

- 算术运算 (FLOPS)
- 通信：移动数据 ← 算法设计尽量减少通信
 - 内存的不同层次之间 (串行算法)
 - 网络中的不同处理器之间 (并行算法)



5.2 减少通信的重要性

■ 算法运行时间可描述为3项：

- $\#flops * time_per_flop$
- $\#words\ moved / bandwidth$
- $\#messages * latency$

■ $Time_per_flop \ll 1/bandwidth \ll latency$

Annual improvements			
Time_per_flop		Bandwidth	Latency
59%	Network	26%	15%
	DRAM	23%	5%

■ 目标：重组计算流程，减少通信

- 所有内存层次间：L1 $\longleftrightarrow$ L2 $\longleftrightarrow$ DRAM $\longleftrightarrow$ network
- 不只是隐藏通信（与计算重叠），可能达到很高加速比

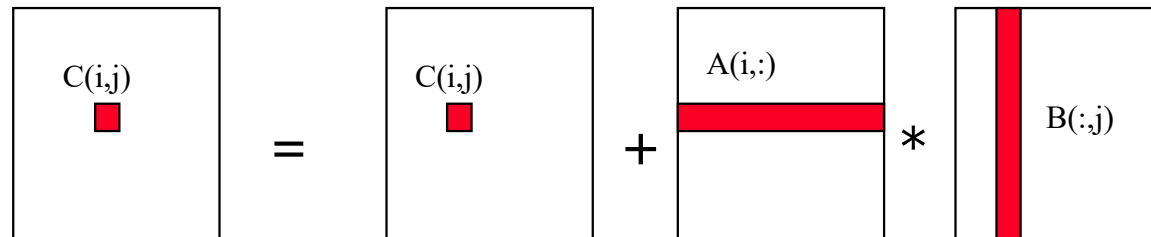
Slide source: Jim Demmel, CS267

5.3 串行矩阵乘法 $C = C + A * B$

```

for i = 1 to n
  {将A的第i行读入缓存,  $n^2$  次读}
  for j = 1 to n
    {将C(i,j)读入缓存,  $n^2$  次读}
    {将B的第j列读入缓存,  $n^3$  次读}
    for k = 1 to n
       $C(i,j) = C(i,j) + A(i,k) * B(k,j)$ 
    {将C(i,j)写回内存,  $n^2$  次写}
  
```

共 $n^3 + 3n^2$ 次读/写



Slide source: Jim Demmel, CS267

5.4 分块矩阵乘法减少通信

■ 分块矩阵乘法 $C=A \cdot B$ ，元素乘/加 $\rightarrow$ 子矩阵乘/加，子矩阵规模依赖于 cache 大小

- $A^{n \times n}, B^{n \times n}, C^{n \times n}$ 分解为 $b \times b$ 的子矩阵，标记为 $A(i,j) \dots$

- b 的选择——3个 $b \times b$ 的子矩阵可放入 cache 中

```
for i = 1 to n/b    for j=1 to n/b    for k=1 to n/b
    C(i,j) = C(i,j) + A(i,k)·B(k,j)    ...  b x b 的矩阵乘法, 4b2 次读/写
```

- 共 $(n/b)^3 \cdot 4b^2 = 4n^3/b$ 次读/写

- 最小化条件： b 尽量大，但子矩阵能放入 cache $3b^2 = \text{cache 大小 } M$ ， $O(n^3/M^{1/2})$

- 多个内存层次 (L1、L2、...) 时会怎样？ 把 $b = (M/3)^{1/2}$ 带入 $4n^3/b$ ，就是 $O(n^3/M^{1/2})$

➤ 每个层次可能都需要多出3层循环

Slide source: Jim Demmel, CS267

5.5 分块 vs Cache-Oblivious算法

■ 分块矩阵乘法 $C=A \cdot B$ ，元素乘/加 $\rightarrow$ 子矩阵乘/加，子矩阵规模依赖于cache大小

- $A^{n \times n}, B^{n \times n}, C^{n \times n}$ 分解为 $b \times b$ 的子矩阵，标记为 $A(i,j) \dots$

- b 的选择——3个 $b \times b$ 的子矩阵可放入cache中

```
for i = 1 to n/b    for j=1 to n/b    for k=1 to n/b
    C(i,j) = C(i,j) + A(i,k)·B(k,j)    ... b x b的矩阵乘法, 4b2 次读/写
```

➤ 多一个内存层次就需要多出3层循环

■ Cache-Oblivious方法： $C=A \cdot B$ 与cache无关

Function $C = \text{RMM}(A, B)$... “R” 表示递归

if A and B are 1×1

$C = A \cdot B$

else ... 将 $A^{n \times n}, B^{n \times n}, C^{n \times n}$ 分解为 $(n/2) \times (n/2)$ 子矩阵，标记为 $A(i,j) \dots$

```
for i = 1 to 2,    for j = 1 to 2,    for k = 1 to 2
    C(i,j) = C(i,j) + RMM( A(i,k), B(k,j) )    ... n/2 x n/2 矩阵乘法
```

Slide source: Jim Demmel, CS267

5.6 通信开销下界

- 假定 n^3 次算术运算
- 串行算法，需快速存储空间大小为M
 - #words moved下界为 $\Omega(n^3 / M^{1/2})$ [Hong, Kung, 81]
 - 用分块算法或Cache-Oblivious算法可达到
- P个处理器上的并行算法
 - 假设每个处理器内存为M，算法负载均衡
 - #words moved下界为 $\Omega(n^3 / (p \cdot M^{1/2}))$ [Irony, Tiskin, Toledo, 04]
 - 若 $M = 3n^2/p$ （每个矩阵一份副本，均匀散布在p个处理器），则下界为 $\Omega(n^3 / p^{1/2})$
 - SUMMA算法、Cannon算法可以达到

Slide source: Jim Demmel, CS267

5.7 适用于所有“直接”线性代数运算的新下界

令 M = 每个处理器中“快速”存储大小
 = cache 大小 (串行) 或 $O(n^2/p)$ (并行)

#flops = 每个处理器完成的浮点运算数, 则

每处理器 #words moved = $\Omega(\text{\#flops} / M^{1/2})$

每处理器 #messages sent = $\Omega(\text{\#flops} / M^{3/2})$

- 矩阵乘、BLAS、LU、QR、eig、SVD、张量约缩、...
- 若干这些操作组成的程序, 如计算 A^k
- 稠密及稀疏矩阵 ($\text{\#flops} \ll n^3$)
- 串行和并行算法
- 某些图论算法 (如 Floyd-Warshall 算法)

Slide source: Jim Demmel, CS267

5.7 适用于所有“直接”线性代数运算的新下界

令 M = 每个处理器中“快速”存储大小
 = cache 大小 (串行) 或 $O(n^2/p)$ (并行)

#flops = 每个处理器完成的浮点运算数, 则

每处理器 #words moved = $\Omega(\text{\#flops} / M^{1/2})$

每处理器 #messages sent = $\Omega(\text{\#flops} / M^{3/2})$

■ 串行情况, 稠密 $n \times n$ 矩阵, 因此 $O(n^3)$ flops

- #words moved = $\Omega(n^3 / M^{1/2})$

- #messages_sent = $\Omega(n^3 / M^{3/2})$

■ 并行情况, 稠密 $n \times n$ 矩阵

- 负载均衡, 因此每个处理器 $O(n^3/p)$

- 数据一份拷贝, 均衡分布, 因此每个处理器 $M = O(n^2/p)$

- #words moved = $\Omega(n^3 / p^{1/2})$

- #messages_sent = $\Omega(p^{1/2})$

Slide source: Jim Demmel, CS267

5.8 可否达到下界？

- LAPACK和ScaLPACK等传统实现是否达到？
 - 大部分没有达到
- 是否有其他算法实现能达到？有
- 算法的目标
 - 最小化#words moved
 - 最小化#message sent
 - 需要新的数据结构
 - 对多级内存层次进行优化：Cache-Oblivious算法最简单
 - 当矩阵能放入最快内存层次时尽量减少flops
 - Cache-Oblivious算法并不总是能达到
- 几乎对所有稠密线性代数计算都能达到
 - 目前为止只有少部分实现达到了
 - 只有少数稀疏算法达到了（如Cholesky）

Slide source: Jim Demmel, CS267

5.9 重用和局部性

■ 考虑数据如何访问

- 数据重用
 - 多次使用相同或邻近数据
 - 计算中的固有特征
- 数据局部性
 - 数据重用，且从“快速存储”访问
 - 使用相同数据或相同的数据阐述
- 如果计算存在重用，我们如何获得局部性？
 - 恰当的数据布局
 - 代码的重排转换

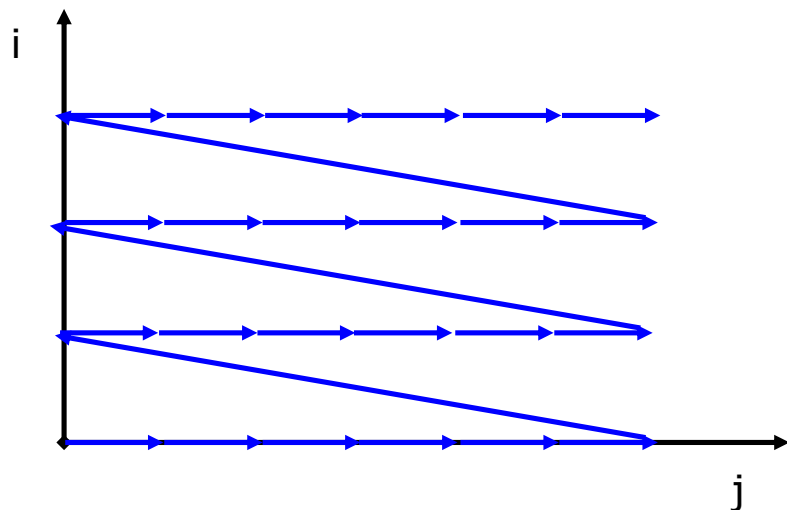
5.10 利用重用：局部性优化

- 循环转换：重排内存访问，提高局部性
- 这些转换方法也适合于并行化
- 两个关键问题
 - 安全性
 - 转换保持了依赖关系？
 - 收益
 - 转换是否能带来收益？
 - 收益是否大于转换开销？

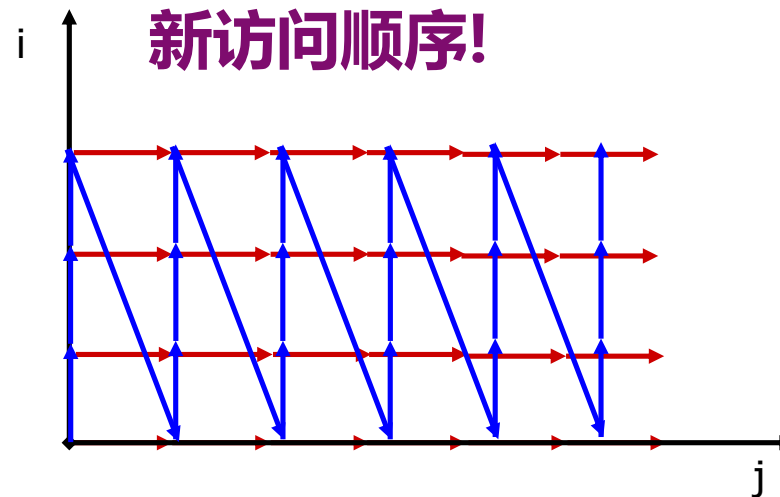
5.11 循环转换：重排

■ 重排循环顺序以改变访存顺序

```
for (i= 0; i<3; i++)  
  for (j=0; j<6; j++)  
    A[i][j+1]=A[i][j]+B[j];
```



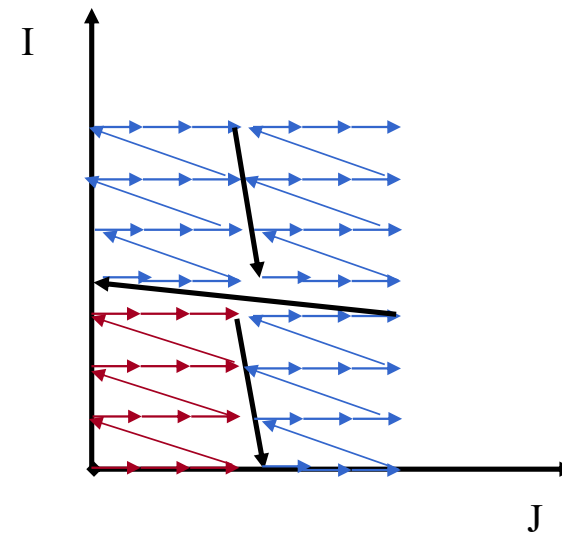
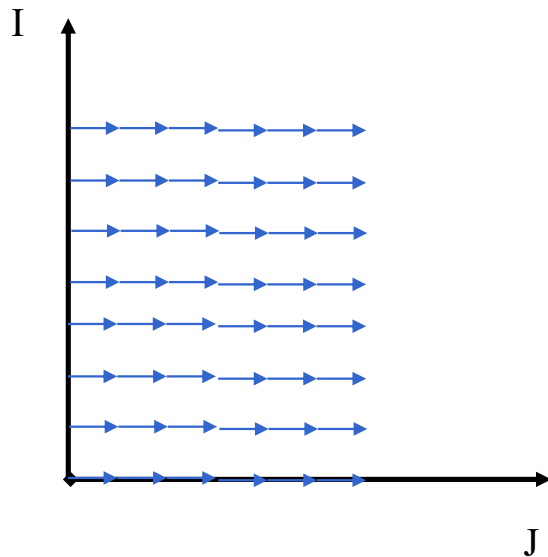
```
for (j=0; j<6; j++)  
  for (i= 0; i<3; i++)  
    A[i][j+1]=A[i][j]+B[j];
```



注意：C/C++多维数组是行主顺序保存，Fortran是列主顺序

5.12 分片（分块）

- 令重用数据的迭代步在时间上更靠近
- 目标是让数据在重用间隔内驻留在高速存储中



5.12 分片示例

```
for (j=1; j<M; j++)
  for (i=1; i<N; i++)
    D[i] = D[i] + B[j,i]
```

Strip mine

```
for (j=1; j<M; j++)
  for (ii=1; ii<N; ii+=s)
    for (i=ii; i<min(ii+s-1,N); i++)
      D[i] = D[i] + B[j,i]
```

Permute

```
for (ii=1; ii<N; ii+=s)
  for (j=1; j<M; j++)
    for (i=ii; i<min(ii+s-1,N); i++)
      D[i] = D[i] + B[j,i]
```

5.13 Unroll——循环展开

- 简单重复语句，重复次数——展开因子
- 只要重复次数不超过迭代数，就安全
- Unroll-and-jam展开外层循环，并重复内层循环的语句（不保证安全）
- 最有效的优化，但过分展开有风险

Original:

```
for (i=0; i<4; i++)
  for (j=0; j<8; j++)
    A[i][j] = B[j+1][i];
```

Unroll j

```
for (i=0; i<4; i++)
  for (j=0; j<8; j+=2)
    A[i][j] = B[j+1][i];
    A[i][j+1] = B[j+2][i];
```

Unroll-and-jam i

```
for (i=0; i<4; i+=2)
  for (j=0; j<8; j++)
    A[i][j] = B[j+1][i];
    A[i+1][j] = B[j+1][i+1];
```

5.13 Unroll——循环展开

```
Unroll j
for (i=0; i<4; i++)
  for (j=0; j<8; j+=2)
    A[i][j] = B[j+1][i];
    A[i][j+1] = B[j+2][i];
```

```
Unroll i
for (i=0; i<4; i+=2)
  for (j=0; j<8; j++)
    A[i][j] = B[j+1][i];
  for (j=0; j<8; j++)
    A[i+1][j] = B[j+1][i+1];
```



```
Unroll-and-jam i
for (i=0; i<4; i+=2)
  for (j=0; j<8; j++)
    A[i][j] = B[j+1][i];
    A[i+1][j] = B[j+1][i+1];
```

5.14 Unroll-and-Jam如何有利局部性

Original:

```
for (i=0; i<4; i++)
  for (j=0; j<8; j++)
    A[i][j] = B[j+1][i] + B[j+1][i+1];
```

Unroll-and-jam i and j loops

```
for (i=0; i<4; i+=2)
  for (j=0; j<8; j+=2) {
    A[i][j]      = B[j+1][i] + B[j+1][i+1];
    A[i+1][j]    = B[j+1][i+1] + B[j+1][i+2];
    A[i][j+1]    = B[j+2][i] + B[j+2][i+1];
    A[i+1][j+1]  = B[j+2][i+1] + B[j+2][i+2];
  }
```

- 短时间内复用B的元素
- 外层循环展开越多效果越明显

5.14 Unroll-and-Jam如何有利局部性

Original:

```
for (i=0; i<4; i++)
  for (j=0; j<8; j++)
    A[i][j] = B[j+1][i] + B[j+1][i+1];
```

Unroll-and-jam i and j loops

```
for (i=0; i<4; i+=2)
  for (j=0; j<8; j+=2) {
    A[i][j]      = B[j+1][i] + B[j+1][i+1];
    A[i+1][j]    = B[j+1][i+1] + B[j+1][i+2];
    A[i][j+1]    = B[j+2][i] + B[j+2][i+1];
    A[i+1][j+1]  = B[j+2][i+1] + B[j+2][i+2];
  }
```

- 更少的循环控制操作
- 无关的计算→指令级并行

5.15 安全性

■ 分片的安全性

- 分片= strip-mine和permutation
 - Strip-mine不重排迭代步
 - Permutation必须合法 或 条纹长度小于依赖距离

■ Unroll-and-Jam的安全性

- Unroll-and-Jam=分片+展开
 - Permutation必须合法 或 展开大小小于依赖距离

5.16 Unroll-and-jam = tile + unroll?

Original:

```
for (i=0; i<4; i++)  
  for (j=0; j<8; j++)  
    A[i][j] = B[j+1][i];
```

Tile i loop:

```
for (ii=0; ii<4; ii+=2)  
  for (j=0; j<8; j++)  
    for (i=ii; i<ii+2; i++)  
      A[i][j] = B[j+1][i];
```

Unroll i tile:

```
for (ii=0; ii<4; ii+=2)  
  for (j=0; j<8; j++)  
    A[ii][j] = B[j+1][ii];  
    A[ii+1][j] = B[j+1][ii+1];
```


目录

1. OpenMP并行模型
2. 并行循环
3. 数据依赖
4. 循环调度
5. 局部性
- 6. 任务并行**

6.1 条件并行

`if` (scalar expression)

- 仅当表达式求值为真时才并行执行
- 否则串行执行

```
#pragma omp parallel if (n > threshold) \  
shared(n,x,y) private(i)  {  
#pragma omp for  
for (i=0; i<n; i++)  
    x[i] += y[i];  
} /*-- End of parallel region --*/
```

Review:
parallel
for
private
shared

6.2 Single和Master结构

- 线程组中只有一个线程执行代码
- 用于I/O或初始化等任务
- 入口或出口无隐式barrier

```
#pragma omp single {  
    <code-block>  
}
```

- 类似的，只有主线程执行代码

```
#pragma omp master {  
    <code-block>  
}
```

6.3 更多的控制机制

```
#pragma omp parallel shared(A,B,C)
{
    tid = omp_get_thread_num();
    A[tid] = big_calc1(tid);
    #pragma omp barrier
    #pragma omp for
        for (i=0; i<N; i++) C[i] = big_calc2(tid);
    #pragma omp for nowait
        for (i=0; i<N; i++) B[i] = big_calc3(tid);
    A[tid] = big_calc4(tid);
}
```

6.4 任务并行

■ 定义

- 同时执行不同计算任务

■ OpenMP支持任务并行

- section：静态划分任务
- task：动态创建任务

■ 任务并行的常见用途：生产者/消费者

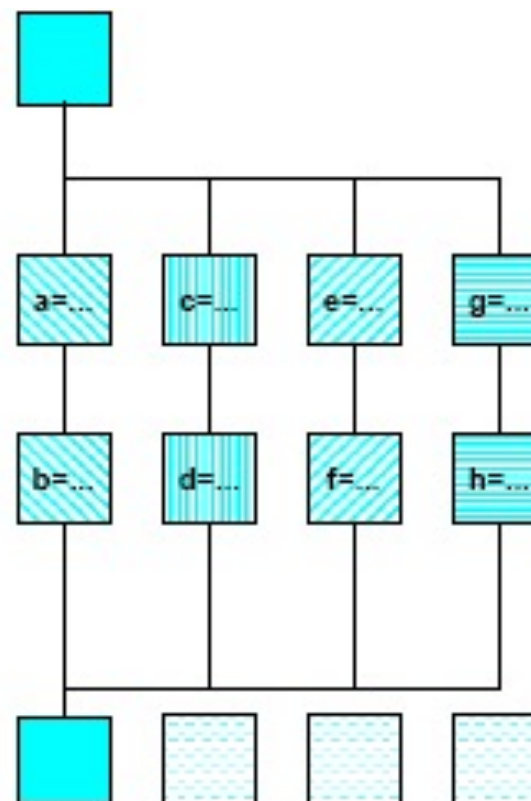
- 生产者创建工作，消费者去处理
- 可理解为一种流水线，类似组装线
- “生产者” 写FIFO队列，“消费者” 读取

6.5 任务并行：section

```

#pragma omp parallel
{
#pragma omp sections
#pragma omp section
    { { a=...;
      b=...; }
#pragma omp section
    { c=...;
      d=...; }
#pragma omp section
    { e=...;
      f=...; }
#pragma omp section
    { g=...;
      h=...; }
} /*omp end sections*/
} /*omp end parallel*/

```



6.5 任务并行：section

```
#pragma omp parallel shared(n,a,b,c,d) private(i)
{
    #pragma omp sections nowait      {
        #pragma omp section
            for (i=0; i<n; i++)
                d[i] = 1.0/c[i];
        #pragma omp section
            for (i=0; i<n-1; i++)
                b[i] = (a[i] + a[i+1])/2;
    } /*-- End of sections --*/
} /*-- End of parallel region
```

6.6 任务例：链表遍历

```
.....  
while (my_pointer) {  
    (void) do_independent_work (my_pointer);  
    my_pointer = my_pointer->next ;  
} // End of while loop  
.....
```

■ 如何用并行for表达？

- 迭代次数必须固定
- 循环不变的循环条件，不能提前退出

6.6 任务并行：task

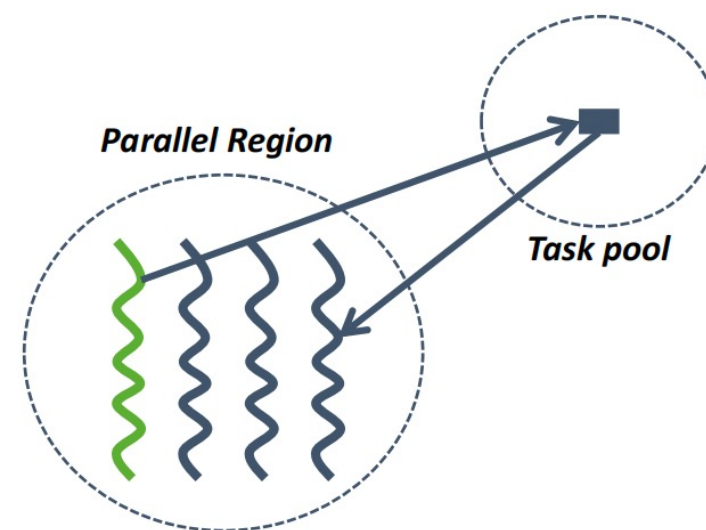
OpenMP 3.0引入了Task（任务）

■ 一个任务具有

- 要执行的代码
- 数据环境（共享的、私有的、归约的）
- 使用数据执行代码的线程

■ 两个动作：打包和执行

- 每个线程打包一个新的任务
- 线程组中某个线程随后执行任务



6.6 任务并行 : task

```

my_pointer = listhead;
#pragma omp parallel {
    #pragma omp single nowait {
        while (my_pointer) {
            #pragma omp task firstprivate(my_pointer) {
                (void) do_independent_work (my_pointer);
            }
            my_pointer = my_pointer->next ;
        }
    } // End of single - no implied barrier (nowait)
} // End of parallel region - implied barrier here

```

while循环由单线程串行执行
 不等待任务完成就遍历下一个结点
 创建出的任务由其他线程并发执行

- **firstprivate**: 从全局变量拷贝初始值到私有变量
- **lastprivate**: 将私有变量值拷贝回全局变量

6.6 任务并行 : task

```

#pragma omp parallel num_threads(4)
{
    int tid = omp_get_thread_num();
    #pragma omp single ← 只在一个线程中创建任务
    {
        printf("[Thread %d] Enter single region\n", tid);
        for (int i = 0; i < 10; i++) {
            #pragma omp task firstprivate(i, tid) ← 打包 (创建) 任务
            {
                int exec_tid = omp_get_thread_num();
                printf("    [Task %d] Executing on thread %d (created by thread %d)\n",
                    i, exec_tid, tid);
            }
            printf("[Thread %d] Task %d packaged (task created)\n", tid, i);
        }
        printf("[Thread %d] All tasks created, now waiting...\n", tid);
        #pragma omp taskwait ← 等待所有任务执行完成
        printf("[Thread %d] All tasks finished.\n", tid);
    }
}

```

6.6 任务并行 : task

```
[Thread 0] Enter single region
[Thread 0] Task 0 packaged (task created)
    [Task 0] Executing on thread 2 (created by thread 0)
[Thread 0] Task 1 packaged (task created)
[Thread 0] Task 2 packaged (task created)
    [Task 1] Executing on thread 3 (created by thread 0)
[Thread 0] Task 3 packaged (task created)
[Thread 0] Task 4 packaged (task created)
[Thread 0] Task 5 packaged (task created)
[Thread 0] Task 6 packaged (task created)
[Thread 0] Task 7 packaged (task created)
[Thread 0] Task 8 packaged (task created)
[Thread 0] Task 9 packaged (task created)
[Thread 0] All tasks created, now waiting...
    [Task 9] Executing on thread 0 (created by thread 0)
    [Task 8] Executing on thread 0 (created by thread 0)
    [Task 2] Executing on thread 1 (created by thread 0)
    [Task 3] Executing on thread 3 (created by thread 0)
    [Task 6] Executing on thread 3 (created by thread 0)
    [Task 4] Executing on thread 2 (created by thread 0)
    [Task 5] Executing on thread 1 (created by thread 0)
    [Task 7] Executing on thread 0 (created by thread 0)
[Thread 0] All tasks finished.
```

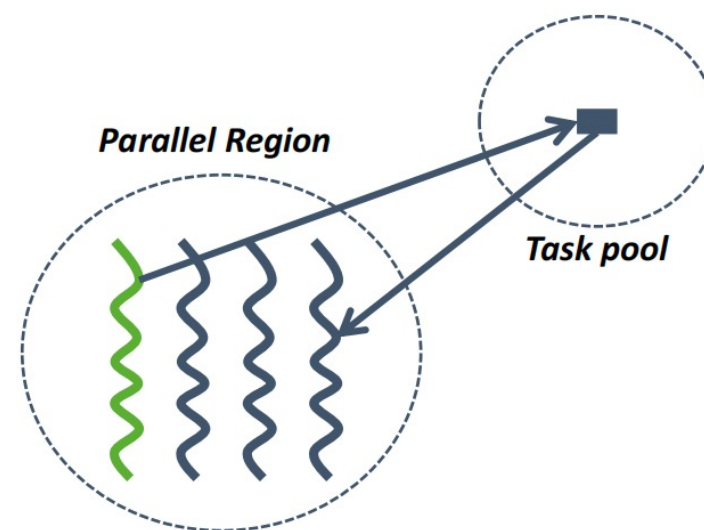
6.6 任务并行：task

■ 传统方法（for/sections）

- 静态划分，编译时确定任务数
- 立即执行

■ Task机制（OpenMP 3.0）

- 动态创建，运行时按需生成
- 延迟执行，自动负载均衡



6.7 简单的生产者-消费者例子

```
// PRODUCER: initialize A with random data
void fill_rand(int nval, double *A) {
    for (i=0; i<nval; i++) A[i] = (double)
rand()/1111111111;
}
```

```
// CONSUMER: Sum the data in A
double Sum_array(int nval, double *A) {
    double sum = 0.0;
    for (i=0; i<nval; i++) sum = sum +
A[i];
    return sum;
}
```

6.7 生产者—消费者模型：关键问题

- 生产者需通知消费者数据已就绪
- 消费者需要等待直至数据就绪
- 生产者和消费者需要一种方式交互数据
 - 生产者的输出是消费者的输入
- 通常通过先进先出（FIFO）队列交互

6.8 生产者—消费者模型：队列读取

- FIFO队列放在全局内存中，被线程们共享
- 如何确保数据更新？
- 需要一种结构保证内存的一致视图
 - 刷新：确保数据写回全局内存

Example:

```
Double A;  
A = compute();  
Flush(A);
```


6.8 生产者—消费者模型：队列读取 Flush

```
flag = 0;  
#pragma omp parallel  
{  
    #pragma omp section  
    {  
        fillrand(N,A);  
        #pragma omp flush  
        flag = 1;  
        #pragma omp flush(flag)  
    }  
}
```

生产者：生成随机数数组

```
#pragma omp section  
{  
    while (!flag)  
        #pragma omp flush(flag)  
    #pragma omp flush  
    sum = sum_array(N,A);  
}  
}
```

消费者：数组求和

6.8 生产者—消费者模型：队列读取 Flush

■ flush **不带参数时**

- 指出所有线程对所有共享对象都有一致的内存视图

■ flush(var)

- 只刷新共享变量 "var"

6.9 生产者—消费者模型：消息传递

- 例子：在共享内存系统实现消息传递
- FIFO队列保存消息
- 线程显式发送/接收消息
 - 发送消息：队列入队
 - 接收消息：出队
 - 确保安全访问

6.9 生产者—消费者模型：消息传递

- 每个线程有独立的链表队列
- 每个线程只从自己的队列取消息（ Dequeue ）
- 任何线程都可以向任何队列发消息（ Enqueue ）

每个线程的伪代码形式：

```
for (sent_msgs = 0; sent_msgs < send_max; sent_msgs++) {  
    Send_msg();  
    Try_receive();  边发送、边接收  
}
```

```
while (!Done())  
    Try_receive();  接收消息，直到所有都完成
```

6.14 生产者—消费者模型：消息传递（发送）

Send_msg()函数的伪代码形式

```
mesg = random();  
dest = random() % thread_count;  
# pragma omp critical  
    Enqueue(queue, dest, my_rank, mesg);
```

■ 使用同步机制更新FIFO队列

6.14 生产者—消费者模型：消息传递（接收）

Try_receive()函数的伪代码形式

```
if (queue_size == 0) return;
else if (queue_size == 1)
#   pragma omp critical
    Dequeue(queue, &src, &mesg);
else
    Dequeue(queue, &src, &mesg);
Print_message(src, mesg);
```

- 只有这个线程从队首取消息
- 其他线程向队尾添加消息
- 因此，尽在尾元素处才需同步

6.14 生产者—消费者模型：消息传递（终止检测）

Done()函数的伪代码形式

```
queue_size = enqueued - dequeued;  
if (queue_size == 0 && done_sending == thread_count)  
    return TRUE;  
else  
    return FALSE;
```

每个线程在自己的工作完成后
都递增此变量

■ 对done_sending需要更多同步操作

OpenMP小结

■ 优点

- 简单修改串行程序即可得到并行程序
- 不必关心底层映射细节
- 可移植、可扩展、单处理器也正确

■ 缺点

- 从头写并行程序的话不是很自然
- 表达常见并行结构并不总是可行
- 局部性处理
- 性能控制